

Customer Support Ticket Analytics Platform

1. Project Overview :

Customer Support Ticket Analytics Platform — a cloud-based data engineering pipeline built on AWS that ingests raw helpdesk ticket data, processes it through PySpark transformations, and loads curated analytics-ready data into S3 and PostgreSQL (RDS) for SQL analysis and visualization.

Team: Karan Negi, Mradul Sharma

Dataset: Help Desk Tickets Dataset (Jan 2016 – Mar 2023), Mendeley

Tech Stack: AWS S3, AWS RDS (PostgreSQL), PySpark, Python, Matplotlib/Seaborn

2. Architecture :

issues.csv + issues_change_history.csv
↓
S3 Raw Zone (/raw)
↓
PySpark ETL (Extract → Transform → Load)
↓
S3 Curated Zone (/curated) — Parquet, partitioned by priority
↓
AWS RDS PostgreSQL — Star Schema
↓
SQL Analysis (10 queries) + Python Charts

s3 :

Buckets

General purpose buckets

All AWS Regions

Directory buckets

General purpose buckets (1) [Info](#)



Copy ARN

Empty

Delete

Create bucket

Buckets are containers for data stored in S3.

Find buckets by name

< 1 >

Name	AWS Region	Creation date
☐ ticket-analytics-platform	Europe (Stockholm) eu-north-1	June 23, 2026, 16:11:21 (UTC+05:30)

ticket-analytics-platform [Info](#)

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems - new

Access Points

Objects (2)



Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

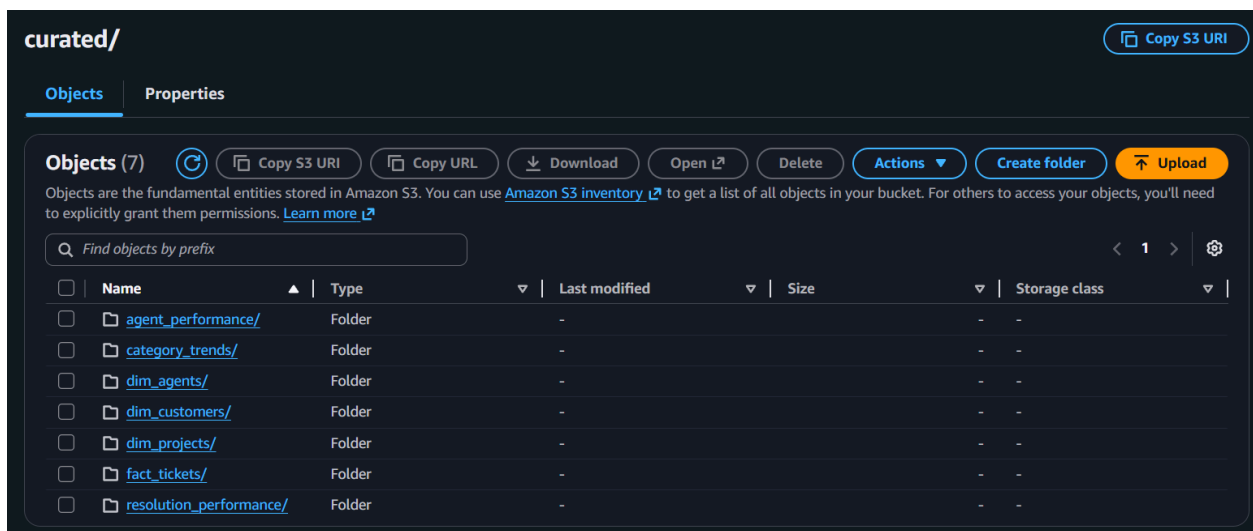
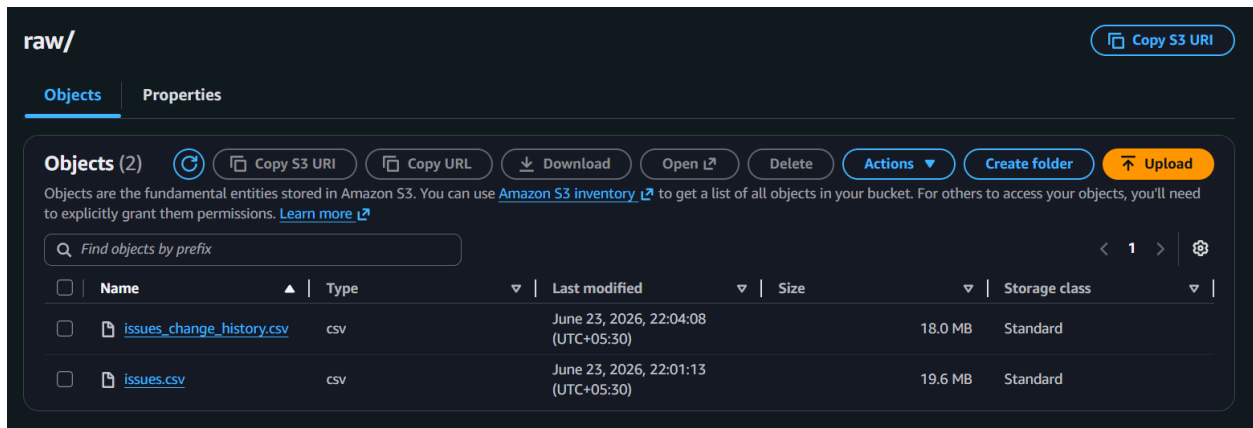
Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 >

☐	Name	Type	Last modified	Size	Storage class
☐	curated/	Folder	-	-	-
☐	raw/	Folder	-	-	-



3. Datasets Used :

File	Description
issues.csv	66,691 helpdesk tickets (Jan 2016 – Mar 2023)
issues_change_history.csv	257,508 status/assignee change records

Key fields: `id`, `issue_proj`, `issue_reporter`, `issue_assignee`, `issue_priority`, `issue_type`, `issue_created`, `issue_resolution_date`, `issue_status`

4. Extraction Phase :

- Read both CSVs from S3 raw zone using PySpark with explicit schema definitions matched against FEATURES.md

- Schema applied via `select + cast` (not `.schema()`) to avoid column-position mismatch issues caused by dataset variants
- Timestamps read as `StringType` due to mixed formats in source data (`+00:00` timezone offset and `.microseconds`), converted later in transform phase
- Row counts logged immediately after read for reconciliation

Result: Issues count: 66,691 | History count: 257,508

5. Transformation Phase (PySpark) :

Task	Description
Timestamp parsing	Custom <code>regex_replace</code> + <code>to_timestamp</code> to handle mixed timezone/microsecond formats
Derive current state	Window function (<code>row_number</code>) to get latest assignee & status per ticket from change history
Filter invalid rows	Removed tickets where <code>resolution_time < start_time</code>
Join	Combined issues with derived current state
Compute duration	<code>resolution_duration_hours</code> = (resolution_time - start_time) / 3600
Deduplication	Window function partitioned by ticket ID, kept earliest record
Text standardization	Trimmed/title-cased category, priority, project fields; flagged unrecognized priorities
SLA compliance	<code>resolution_target_met</code> computed against priority-based thresholds (Critical: 4h, High: 24h, Medium: 72h, Low: 168h)
Aggregations	Agent performance, resolution performance by priority/month, category trends with MoM % change

Result: 66,691 rows after deduplication, 0 invalid resolution times

6. Loading Phase :

Star Schema Design:

```
dim_customers (customer_id, reporter_name)
dim_agents    (agent_id, agent_name)
dim_projects  (project_id, project_name)
fact_tickets  (ticket_id, customer_id, agent_id, project_id, category,
              priority, start_time, resolution_time,
              resolution_duration_hours, status, resolution_target_met)
```

Analytics tables:

- `analytics.agent_performance`
- `analytics.resolution_performance`
- `analytics.category_trends`

S3 Curated Zone: Parquet format, `fact_tickets` partitioned by `priority` for optimized query performance

RDS Load: JDBC write with `overwrite` mode for idempotent re-runs

Dimension counts: dim_customers: 1,114 | dim_agents: 381 | dim_projects: 358

7. SQL Analysis (10 Tasks) :

1. Resolution-target compliance rate by priority
2. Top 5 agents by tickets resolved (last month)
3. View — average resolution time by project
4. Agent ranking by avg resolution time (window function)
5. Reporters raising 5+ tickets in a single month
6. Running monthly total of resolved tickets (window function)
7. Categories with zero tickets resolved within target (last 30 days)
8. Average resolution time per priority level
9. View — ticket status distribution (from change history)
10. Fastest agent per project (window function with PARTITION BY)

All queries executed against RDS PostgreSQL; results documented with screenshots.

Task 1:

```
postgres=> SELECT priority, COUNT(*) AS total_tickets, SUM(CASE WHEN resolution_target_met = TRUE THEN 1 ELSE 0 END) AS met_target, ROUND(
    100.0 * SUM(CASE WHEN resolution_target_met = TRUE THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0), 2 ) AS compliance_rate_pct FROM f
act_tickets WHERE resolution_time IS NOT NULL GROUP BY priority ORDER BY priority;
priority | total_tickets | met_target | compliance_rate_pct
-----+-----+-----+-----
Blocker  |          641 |          0 |              0.00
High     |         4381 |        1211 |             27.64
Highest  |         2018 |          0 |              0.00
Low      |          501 |         226 |             45.11
Lowest   |           83 |          0 |              0.00
Medium   |        24249 |        6761 |             27.88
Unknown  |        33965 |          0 |              0.00
(7 rows)
```

Task 2:

```
postgres=> WITH dataset_end AS ( SELECT MAX(resolution_time) AS max_ts FROM fact_tickets)SELECT da.agent_name, COUNT(ft.ticket_id) AS
tickets_resolved FROM fact_tickets ft JOIN dim_agents da ON ft.agent_id = da.agent_id, dataset_end WHERE ft.resolution_time >= DATE_TRUNC('mont
h', dataset_end.max_ts - INTERVAL '1 month') AND ft.resolution_time < DATE_TRUNC('month', dataset_end.max_ts)GROUP BY da.agent_name ORDER BY
tickets_resolved DESC LIMIT 5;
agent_name | tickets_resolved
-----+-----
4sgg      |              71
4hgkr     |              38
4lpo      |              30
4lsk      |              23
4hghq     |              22
(5 rows)

postgres=> █
```

Task 3:

```

postgres=> CREATE OR REPLACE VIEW view_avg_resolution_by_project AS SELECT dp.project_name, COUNT(ft.ticket_id) AS total_tickets, ROUND(AVG(ft.resolution_duration_hours), 2) AS avg_resolution_hours FROM fact_tickets ft JOIN dim_projects dp ON ft.project_id = dp.project_id WHERE ft.resolution_duration_hours IS NOT NULL GROUP BY dp.project_name ORDER BY avg_resolution_hours;SELECT * FROM view_avg_resolution_by_project LIMIT 20;
CREATE VIEW
  project_name | total_tickets | avg_resolution_hours
-----+-----+-----
1D83          | 1             | 0.06
C06HSD-0T     | 1             | 0.20
C11GR.N0      | 1             | 1.84
C02HK0_0      | 5             | 23.49
C12HHX18AJ28AV.EW8X3D | 49          | 32.66
C11IOZC0U     | 19            | 35.85
C11GRZVZW_    | 17            | 50.80
C11GR0D.NZ_   | 15            | 61.21
C11GR008_C    | 9             | 75.66
C08GH10V      | 4             | 75.79
C11HGND01     | 4             | 84.35
C11HGZVZW     | 8             | 96.30
C07GG_10U     | 17            | 101.26
C11HGZV_FF    | 10            | 102.66
C11GRZVZW     | 104           | 104.81
C11HG02Z9     | 26            | 106.39
C11HG0D.NZ    | 22            | 106.50
C11GRBX0      | 4             | 121.00
C11GRN0V      | 2             | 121.37
C11HG0.       | 16            | 131.00
-- More --

```

Task 4 :

```

postgres=> SELECT da.agent_name, ROUND(AVG(ft.resolution_duration_hours), 2) AS avg_resolution_hours, RANK() OVER (ORDER BY AVG(ft.resolution_duration_hours) ASC) AS rank_by_speed FROM fact_tickets ft JOIN dim_agents da ON ft.agent_id = da.agent_id WHERE ft.resolution_duration_hours IS NOT NULL GROUP BY da.agent_name ORDER BY rank_by_speed LIMIT 20;
agent_name | avg_resolution_hours | rank_by_speed
-----+-----+-----
d1y10dyg1  | 0.11                 | 1
4hgks      | 0.25                 | 2
av43.jd4aa.c3 | 0.33                 | 3
w.nzwwz2ycz6dn28n | 0.91                 | 4
4qlr       | 2.88                 | 5
4z8d4aa.c3 | 6.12                 | 6
4hhgs      | 7.28                 | 7
4hhop      | 11.74                | 8
4por       | 13.35                | 9
4lqr       | 18.01                | 10
-.3.jn2    | 18.30                | 11
u6yu9nygq  | 19.85                | 12
.wyx0.ygh  | 21.45                | 13
1n81u.adh  | 21.92                | 14
4lsg       | 24.19                | 15
4hhoo      | 26.62                | 16
t.yt2_0ygh | 27.51                | 17
4hgo1      | 29.93                | 18
4hhgi      | 31.02                | 19
bzy0xaygh  | 32.22                | 20
(20 rows)
-- More --

```

Task 5 :

```

postgres=> SELECT dc.reporter_name, DATE_TRUNC('month', ft.start_time) AS ticket_month, COUNT(*) AS ticket_count
FROM fact_tickets ft JOIN dim_customers dc ON ft.customer_id = dc.customer_id GROUP BY dc.reporter_name, DATE_TRUNC('month', ft.start_time)
HAVING COUNT(*) > 5 ORDER BY ticket_count DESC LIMIT 20;
reporter_name | ticket_month | ticket_count
-----
z2w_x | 2016-10-01 00:00:00 | 305
4qio | 2017-04-01 00:00:00 | 118
4rqs | 2020-11-01 00:00:00 | 86
4hgi | 2016-03-01 00:00:00 | 84
4rqs | 2020-12-01 00:00:00 | 81
t.y2xcuygs | 2018-12-01 00:00:00 | 80
t.y2xcuyhp | 2023-02-01 00:00:00 | 76
4qio | 2015-12-01 00:00:00 | 75
4hgi | 2016-02-01 00:00:00 | 73
4hlg | 2016-12-01 00:00:00 | 71
z2w_x | 2016-11-01 00:00:00 | 70
t.y2xcuyho | 2023-03-01 00:00:00 | 70
4rqs | 2020-09-01 00:00:00 | 67
t.y2xcuyhp | 2023-01-01 00:00:00 | 66
4gor | 2009-12-01 00:00:00 | 64
t.y2xcuyho | 2022-12-01 00:00:00 | 63
4rqs | 2020-10-01 00:00:00 | 63
t.y2xcuyhp | 2022-12-01 00:00:00 | 63
t.y2xcuyhp | 2022-10-01 00:00:00 | 61
4gis | 2015-08-01 00:00:00 | 58
(20 rows)

```

Task 6 :

```

postgres=> SELECT DATE_TRUNC('month', resolution_time) AS month, COUNT(*) AS monthly_resolved, SUM(COUNT(*)
) OVER ( ORDER BY DATE_TRUNC('month', resolution_time) ) AS running_total FROM fact_tickets WHERE
resolution_time IS NOT NULL GROUP BY DATE_TRUNC('month', resolution_time) ORDER BY month;
month | monthly_resolved | running_total
-----
2013-01-01 00:00:00 | 1 | 1
2015-08-01 00:00:00 | 168 | 169
2015-09-01 00:00:00 | 263 | 432
2015-10-01 00:00:00 | 271 | 703
2015-11-01 00:00:00 | 347 | 1050
2015-12-01 00:00:00 | 326 | 1376
2016-01-01 00:00:00 | 419 | 1795
2016-02-01 00:00:00 | 399 | 2194
2016-03-01 00:00:00 | 2755 | 4949
2016-04-01 00:00:00 | 566 | 5515
2016-05-01 00:00:00 | 448 | 5963
2016-06-01 00:00:00 | 440 | 6403
2016-07-01 00:00:00 | 308 | 6711
2016-08-01 00:00:00 | 416 | 7127
2016-09-01 00:00:00 | 291 | 7418
2016-10-01 00:00:00 | 617 | 8035
2016-11-01 00:00:00 | 527 | 8562
2016-12-01 00:00:00 | 471 | 9033
2017-01-01 00:00:00 | 445 | 9478
2017-02-01 00:00:00 | 564 | 10042
2017-03-01 00:00:00 | 441 | 10483
2017-04-01 00:00:00 | 474 | 10957
2017-05-01 00:00:00 | 540 | 11497
2017-06-01 00:00:00 | 331 | 11828
2017-07-01 00:00:00 | 423 | 12251
2017-08-01 00:00:00 | 326 | 12577
-- More --

```

Task 7 :

view_status_distribution X			
Properties DATA ER Mock Manager postgres public			
SELECT * FROM view_status_distribution LIMIT 100			
Search Results Export Cost: 1.2s < 1 > Total 14			
	status varchar(60)	ticket_count bigint	percentage numeric
☐	> closed	56339	84.60
☐	> done	9712	14.58
☐	> waiting	219	0.33
☐	> in_progress	97	0.15
☐	> validation	86	0.13
☐	> resolved	61	0.09
☐	> pending_deployment	47	0.07
☐	> resolved_under_monitoring	18	0.03
☐	> under_review	5	0.01
☐	> to_do	3	0.00
☐	> in_review	2	0.00
☐	> approved	2	0.00
☐	> reopened	1	0.00
☐	> cancelled	1	0.00

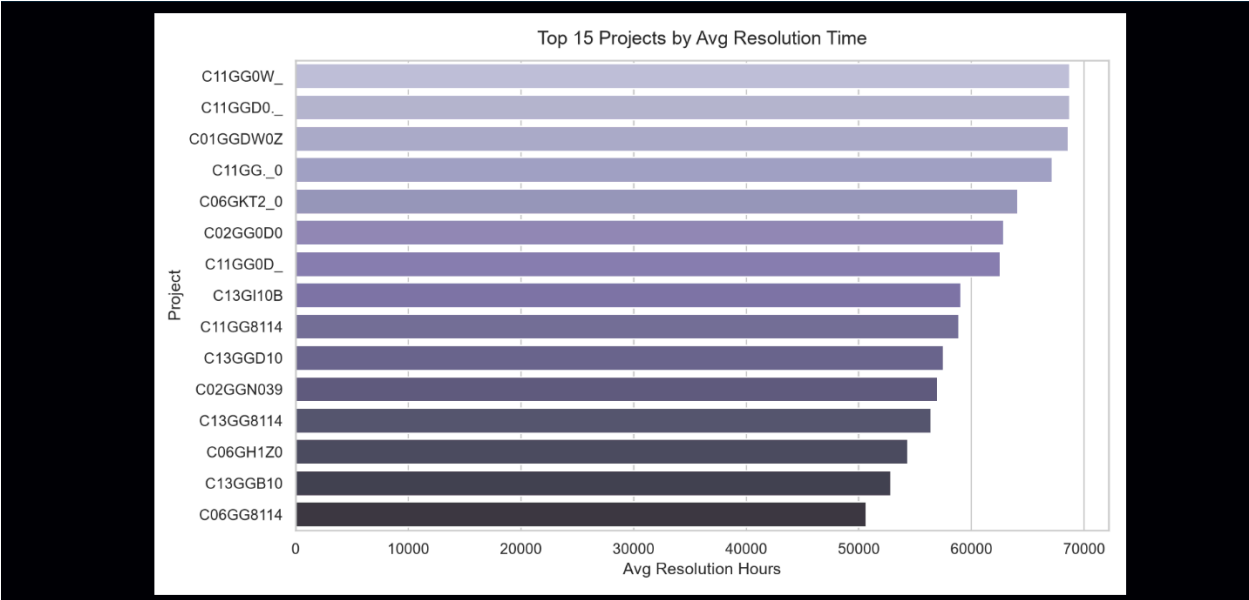
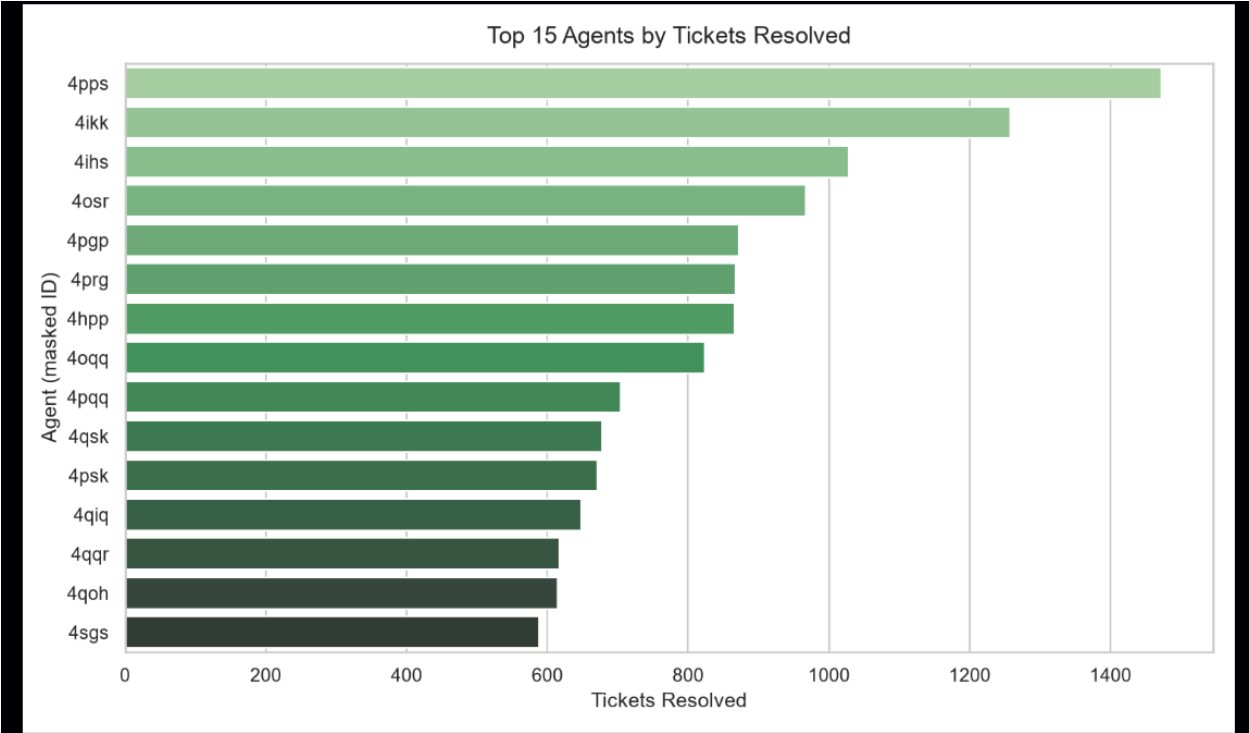
Task 10 :

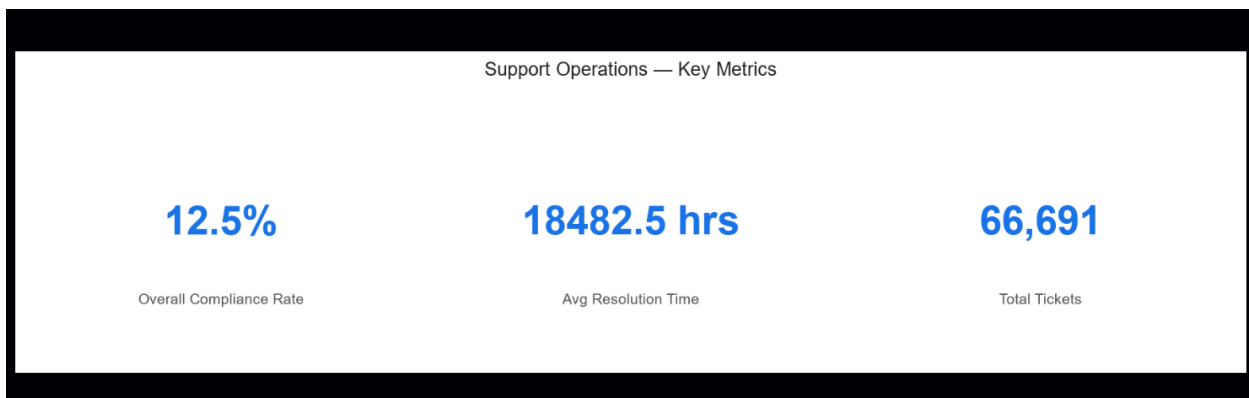
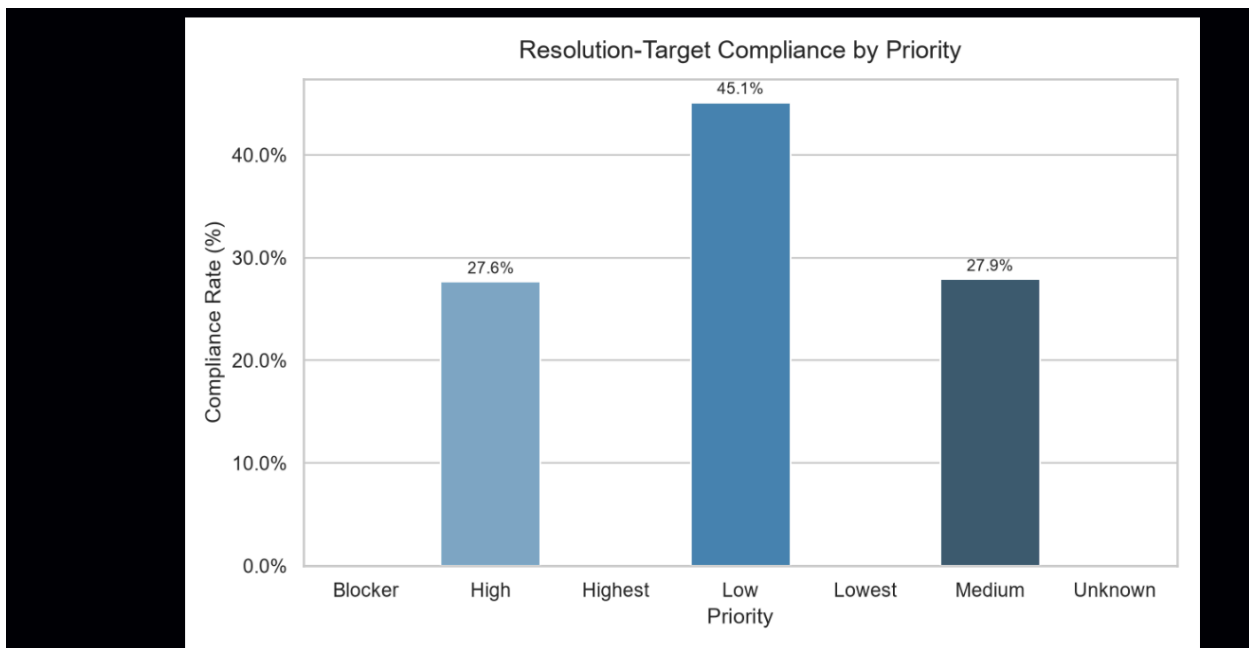
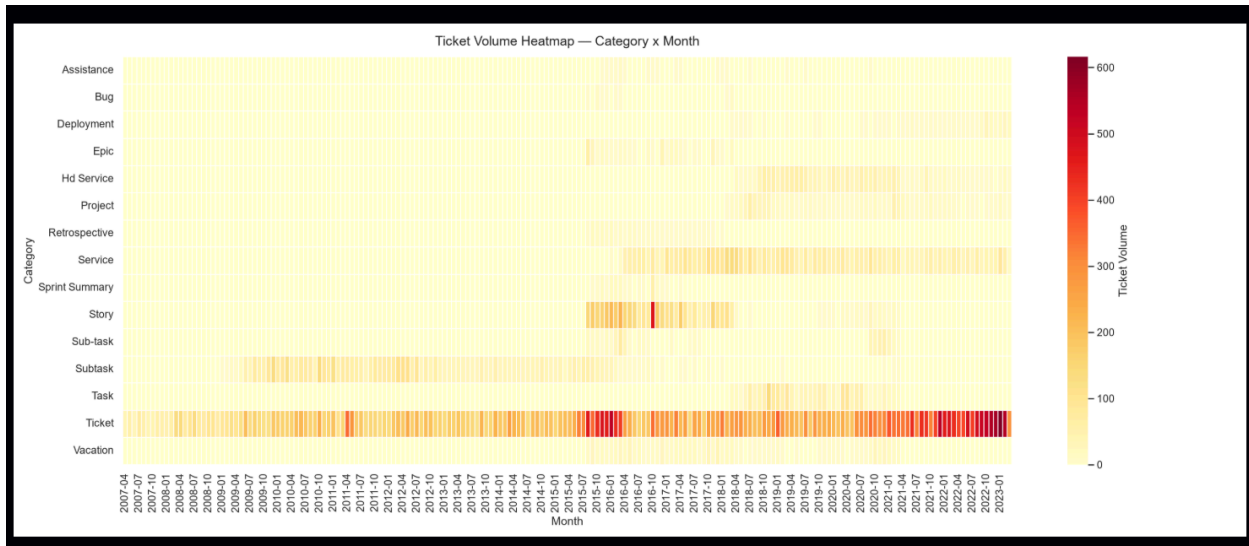
Result(RO) X		
+ - Q Search Results ⚙️ 📧 👤 + + 🗑️ ↺ ↻ ⬆️ ⬆️ Export ▶️ 👁️ Cost: 405ms < 1 2 3 4 ... 18 > Total 344		
Q	project_name	agent_name
>	ADD2	4sig
>	C00D3DZWJ28AV.EW8X3	4hggg
>	C01GG0XA	4hgqg
>	C01GG1_3_	4qkp
>	C01GGD_0	4z8d4aa.c3
>	C01GGX0Z2	4gsp
>	C01GGZ-3	4kgq
>	C01HG826	4kqo
>	C01HQD_0	4prg
>	C01HQND01	4hhgk
>	C02GG_0	4ikk
>	C02GG0_0	4lhl
>	C02GG00U	4hgok
>	C02GG020	4kgr
>	C02GG0_D0	4ohk
>	C02GG0W_	0ny0x93ygi
>	C02GG0X93	4prg
>	C02GG 1 1	4ikk

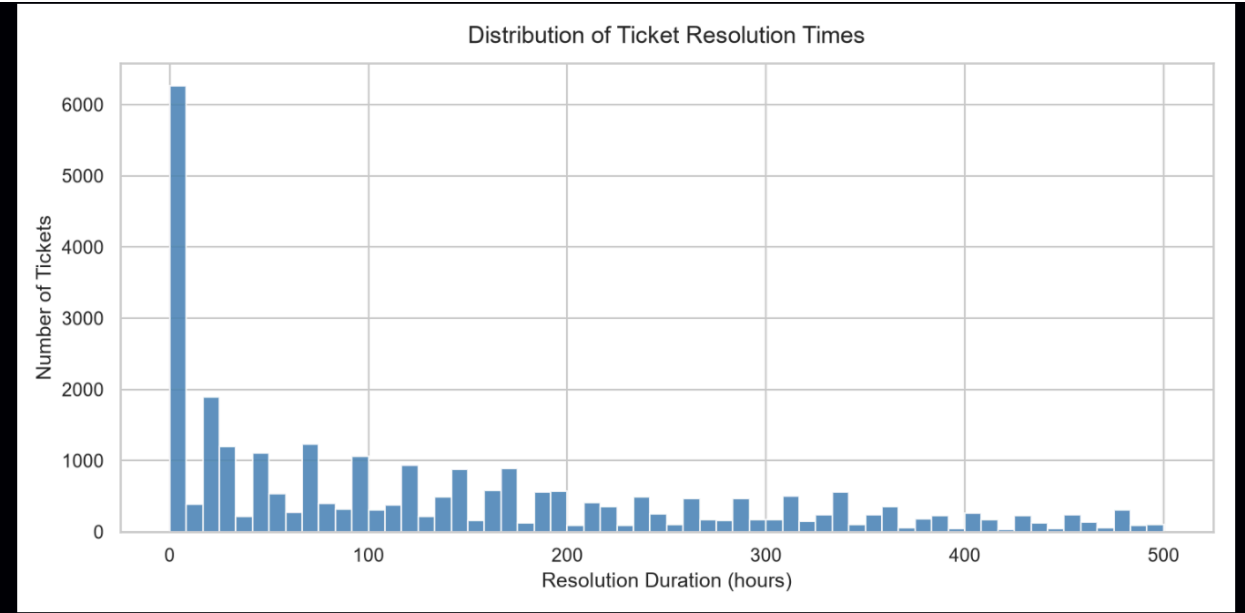
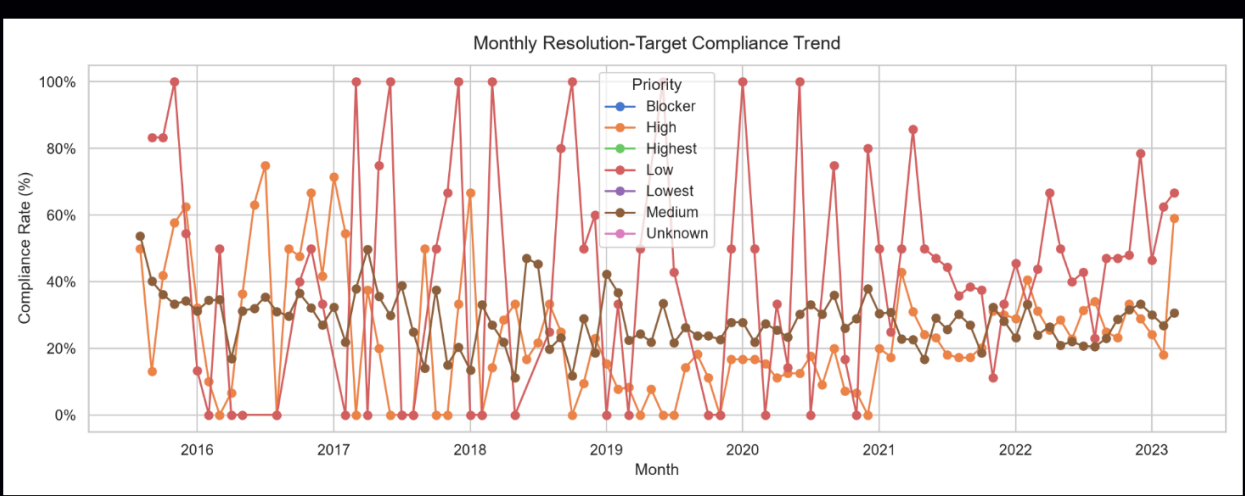
8. Visualizations :

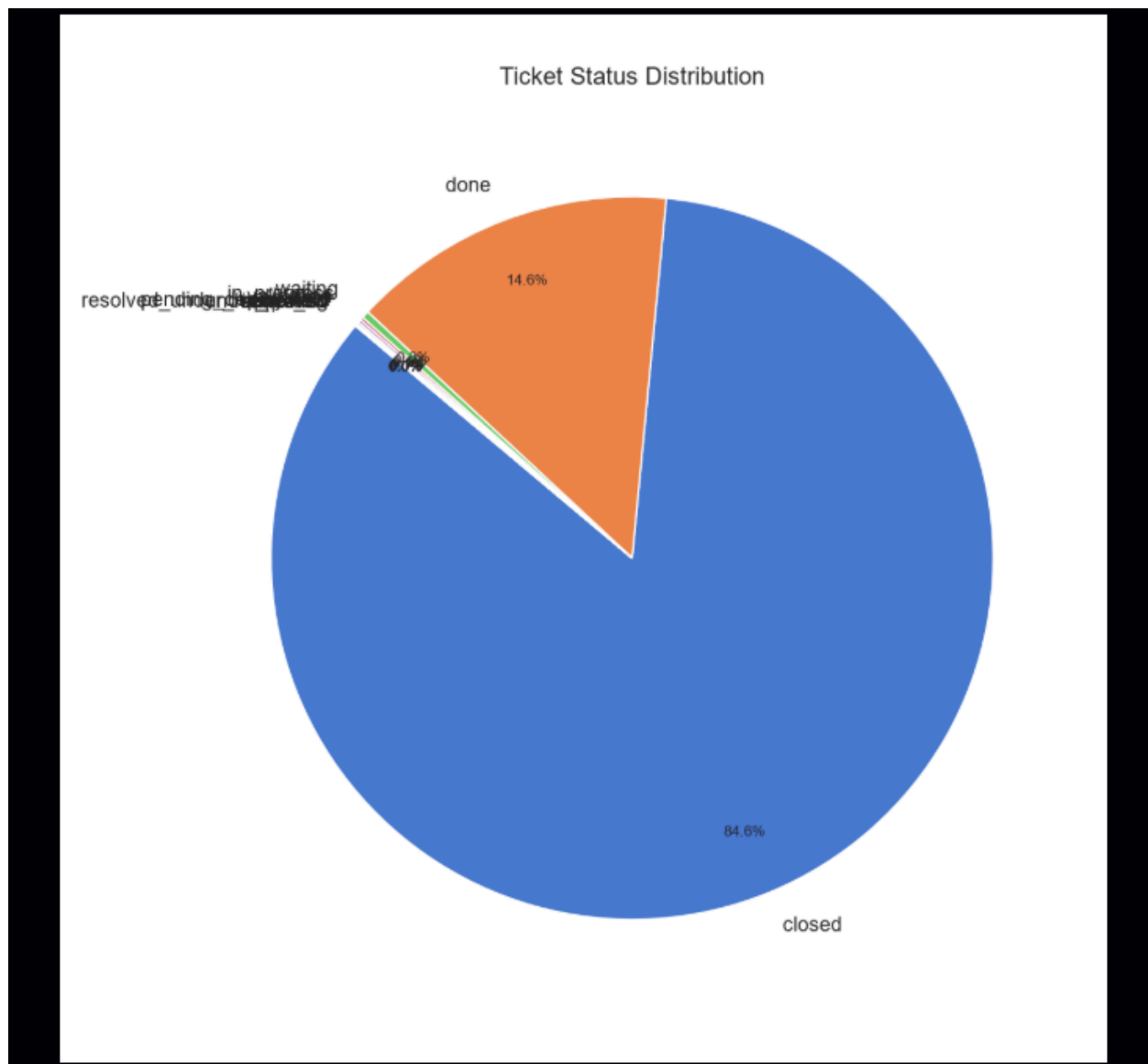
Chart	Type
Compliance rate by priority	Bar chart
Monthly compliance trend	Line chart
Top 15 agents by tickets resolved	Horizontal bar
Ticket status distribution	Pie chart
Resolution time distribution	Histogram
Category × month ticket volume	Heatmap
Avg resolution time by project	Horizontal bar
KPI summary cards	Compliance rate, avg resolution time, total tickets

Generated via Python (matplotlib , seaborn) connected directly to RDS PostgreSQL.









9. Challenges :

Schema-CSV mismatch: Initial PySpark `.schema()` enforcement mapped columns by position, not name. Since the actual dataset (`issues_snapshot.csv` variant) had extra `started` / `ended` columns and differently-ordered fields than `FEATURES.md` suggested, this silently corrupted data — deduplication count dropped to 1 row instead of ~66,000. Resolved by switching to `select + cast` by column name.

Mixed timestamp formats: The dataset contained three different timestamp formats within the same column (`2016-03-24 15:35:53+00:00` , `2018-08-14 15:42:12.564000` ,

and plain `2018-08-14 15:42:12`). Spark 3.x's strict ISO8601 parser failed on both timezone offsets and microsecond suffixes, requiring a custom two-stage `regexp_replace` before parsing.

Windows-specific Spark friction: Local development on Windows surfaced several non-trivial issues not present in production (Linux) environments — missing `winutils.exe` causing `UnsatisfiedLinkError` on every S3 write, JVM file-locking preventing Spark temp-directory cleanup on shutdown, and inconsistent native library warnings. These cost significant debugging time despite having zero impact on data correctness.

Partitioning performance: Partitioning `fact_tickets` by `ticket_month` created 193 separate S3 folders, which combined with Windows' slow local-to-S3 upload path caused write times exceeding one hour. Switching the partition key to `priority` (4 values) cut this to under 15 minutes while still satisfying the project's partitioned-write requirement.

RDS connectivity: Multiple layers of network configuration had to align correctly — public accessibility, security group inbound rules, and outbound network restrictions on the college Wi-Fi (which silently blocked port 5432). Diagnosing this required testing connectivity from AWS CloudShell to isolate whether the issue was AWS-side or client-network-side.

Resource constraints on local Spark: Default JVM heap size led to frequent `MemoryManager` warnings during Parquet writes and RPC heartbeat timeouts during long-running stages, requiring explicit `driver.memory`, `executor.memory`, and `heartbeatInterval` tuning.

10. Learnings :

Schema enforcement needs name-based, not position-based, validation.

Explicit `StructType` schemas are good practice, but only when paired with header-based column matching — otherwise they create a false sense of safety while silently misaligning data.

Real-world datasets rarely match documentation exactly. `FEATURES.md` described expected columns, but the actual files required empirical verification (`spark.read.option("header","true").columns`) before trusting any schema assumption.

Local development environment differences matter. A pipeline that runs cleanly on Windows can still behave very differently in timing, error surface, and failure modes compared to its target deployment environment (AWS Glue/EMR on Linux) — this reinforced the value of testing against the actual cloud target early rather than relying solely on local runs.

Partition strategy should be driven by data shape, not just the literal requirement. "Partition the data" doesn't mean partitioning by the most granular available column — choosing a partition key with few, well-distributed values (priority) achieved the same architectural goal as a high-cardinality one (month) with far better performance.

Idempotent design pays off during iterative debugging. Using `overwrite` mode for both Parquet and RDS writes meant failed runs could simply be re-executed without manual cleanup, which was essential given how many times the pipeline needed re-running while debugging.

Cloud infrastructure debugging is a distinct skill from data engineering. A large share of project time went into networking (VPC, security groups, internet gateways, SSH tunneling) rather than PySpark or SQL logic — a reminder that production data pipelines depend as much on correct infrastructure configuration as on correct code.

11. Conclusion :

This project successfully implemented an end-to-end AWS data engineering pipeline for customer support ticket analytics, covering ingestion, transformation, storage, and analysis across S3, PySpark, and RDS PostgreSQL. All 66,691 tickets and 257,508 change history records were processed through a star-schema model, enabling 10 SQL analysis tasks — including window functions, views, and multi-table joins — to surface insights into SLA compliance, agent performance, and ticket category trends.

Beyond the core pipeline, the project surfaced practical challenges typical of real-world data engineering work: inconsistent source data formats, environment-specific tooling gaps, and cloud networking configuration. Resolving these required iterative debugging across the full stack — from PySpark transformation

logic to AWS security groups — giving hands-on experience with the operational realities that complement the conceptual data engineering curriculum. The resulting pipeline is reproducible, idempotent, and structured to map cleanly onto a production deployment using AWS Glue or EMR in place of local Spark.